UNIVERSIDAD TECNOLÓGICA DE CHILE
INSTITUTO PROFESIONAL
CENTRO DE FORMACIÓN TÉCNICA

Informe Técnico: Proyecto CyberScanner v4.9.9

Asignatura: Redes Avanzadas I / Telecomunicaciones

Integrantes: José Tapia, Martin Cortes, Pedro Roga Carvajal

Fecha: 08 de abril de 2026

Introducción

El presente informe documenta el desarrollo y pruebas de **CyberScanner v4.9.9**, una herramienta de auditoría de red diseñada para la identificación de hosts y servicios en redes locales. El proyecto se enfoca en la eficiencia técnica mediante el uso de multihilos y una interfaz gráfica intuitiva desarrollada en Python.

1. Desarrollo Técnico y Funcionalidades

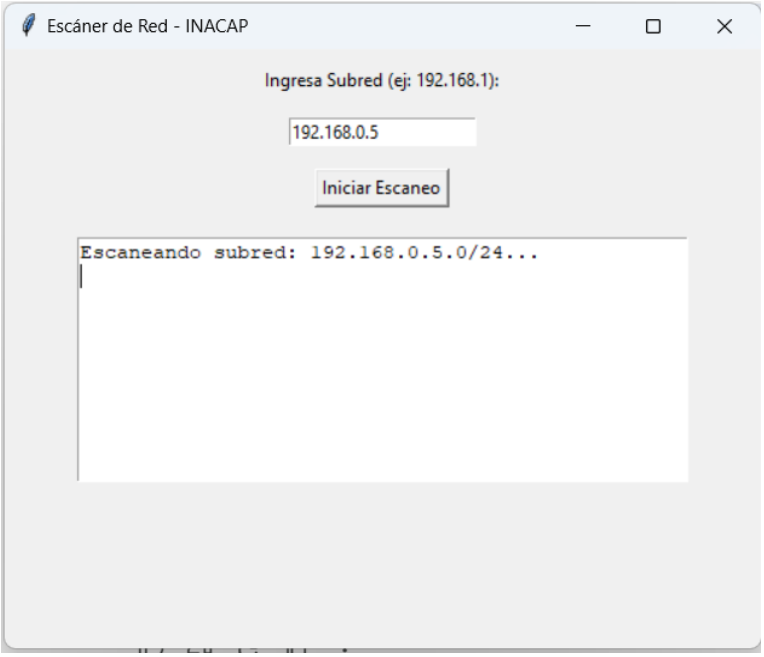
El software opera realizando un análisis exhaustivo en dos capas del modelo OSI:

- Capa 3 (Red): Detección de dispositivos mediante ICMP Echo Request.
- Capa 4 (Transporte): Identificación de servicios TCP abiertos, cerrados o filtrados por firewall.

Optimización: Uso de un semáforo de 40 hilos para evitar la saturación de recursos del sistema.

2. Versiones de prueba y desarrollo del código.

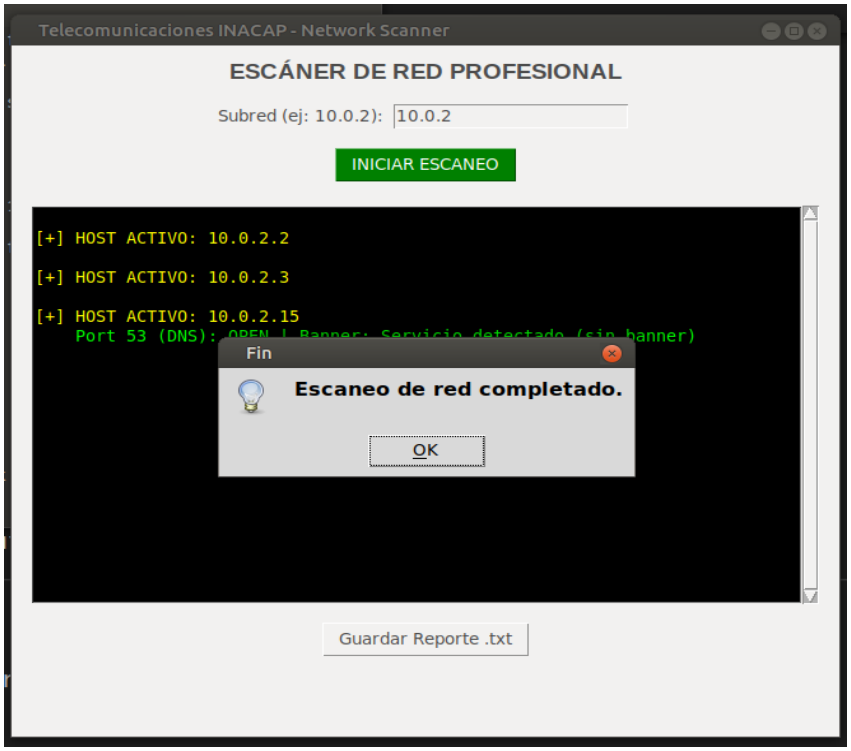
Figura 1: Escáner primeras versiones



Esta captura muestra la primera iteración operativa del software. En esta etapa, el desarrollo se centró exclusivamente en cumplir con la funcionalidad básica requerida

por la pauta: la entrada de un rango de red y la ejecución de un escaneo de Capa 3. Como se observa, el diseño gráfico (GUI) es minimalista e industrial, priorizando la estabilidad del motor de escaneo secuencial sobre los aspectos estéticos.

Figura 2: Versiones mejoras estética y añadido de funciones extras.



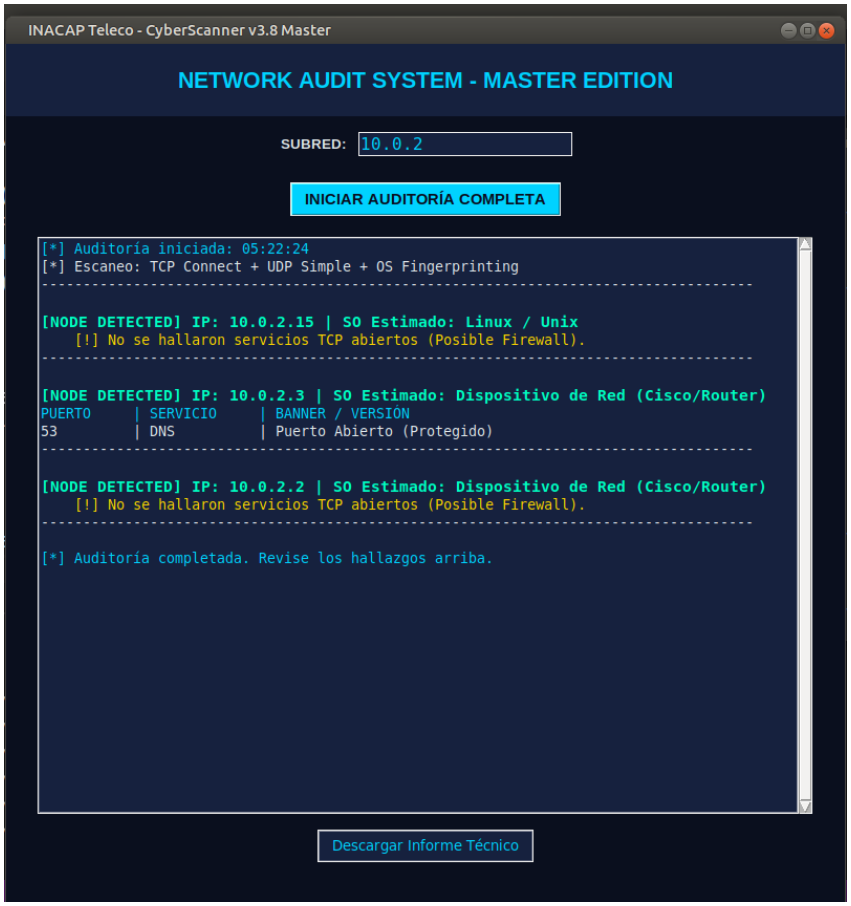
En esta etapa de desarrollo, el equipo integró funciones de identificación de servicios (Banner Grabbing) y una consola de salida con formato de colores para mejorar la legibilidad técnica.

Avance Técnico: Se incorporó la detección de puertos específicos (como el DNS puerto 53) y la capacidad de identificar servicios activos sin banner disponible.

Mejora de UX: Se añadió una ventana emergente de confirmación al finalizar el escaneo y un botón dedicado para la exportación de reportes en formato .txt, facilitando la persistencia de los datos recolectados.

Diseño: Se observa una transición hacia una interfaz más limpia, con encabezados claros y una consola que ya comienza a separar visualmente los hosts encontrados del análisis de sus puertos.

Figura 3: Versiones más avanzada, con colores y funciones completas.



En esta iteración, el desarrollo se enfocó en alcanzar un nivel de análisis técnico avanzado y una identidad visual robusta (Modo Oscuro). La versión presentada destaca por:

Huella Digital del Sistema (OS Fingerprinting): Implementación de lógica para estimar el sistema operativo del nodo detectado (Linux/Unix, Cisco/Router), cumpliendo con uno de los puntos "extra" de la pauta.

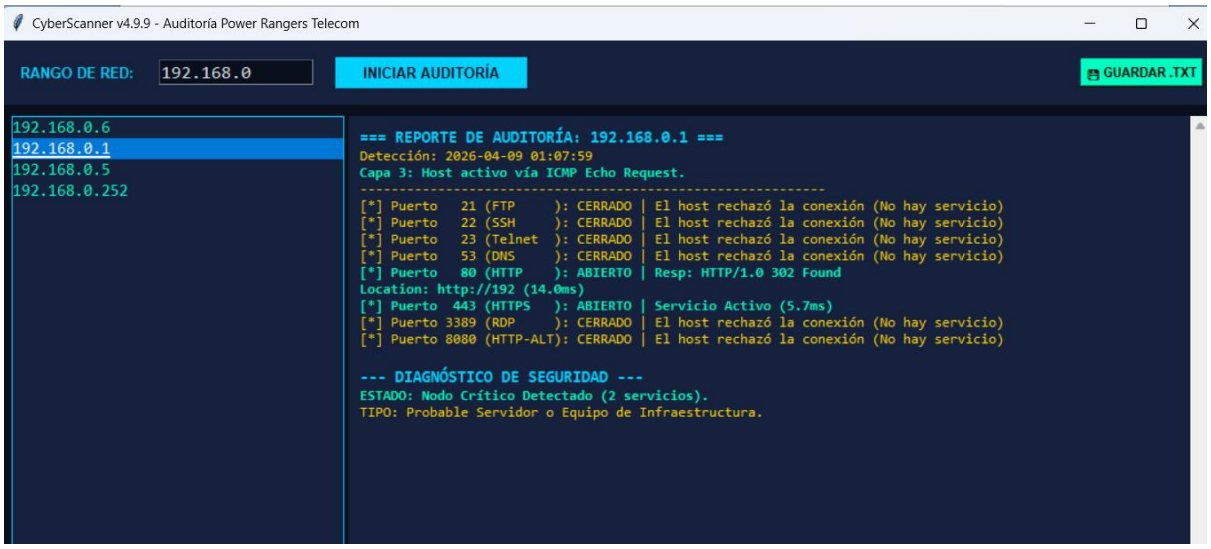
Escaneo Híbrido: Se añadió soporte para escaneos de tipo TCP Connect y UDP Simple, ampliando el espectro de visibilidad sobre la red.

Interfaz Profesional: Mejora significativa en la visual mediante el uso de etiquetas de colores (Verde para nodos detectados, Amarillo para advertencias de firewall) y una consola de texto enriquecida.

Limitación Técnica Identificada: Aunque el escaneo es el más completo hasta ese momento, esta versión aún operaba como script de Python dependiente del entorno de

desarrollo, careciendo de la compatibilidad con el kernel de Windows para su ejecución independiente como .exe

Figura 4: Versión Final



Esta captura muestra el estado definitivo del proyecto, donde se han consolidado todas las funcionalidades técnicas y estéticas bajo una arquitectura de alto rendimiento.

- **Optimización de Capa 4:** El escáner no solo identifica puertos abiertos, sino que calcula la latencia en milisegundos (ms) y realiza una captura de banners (HTTP/1.0 302 Found) para validar servicios reales.

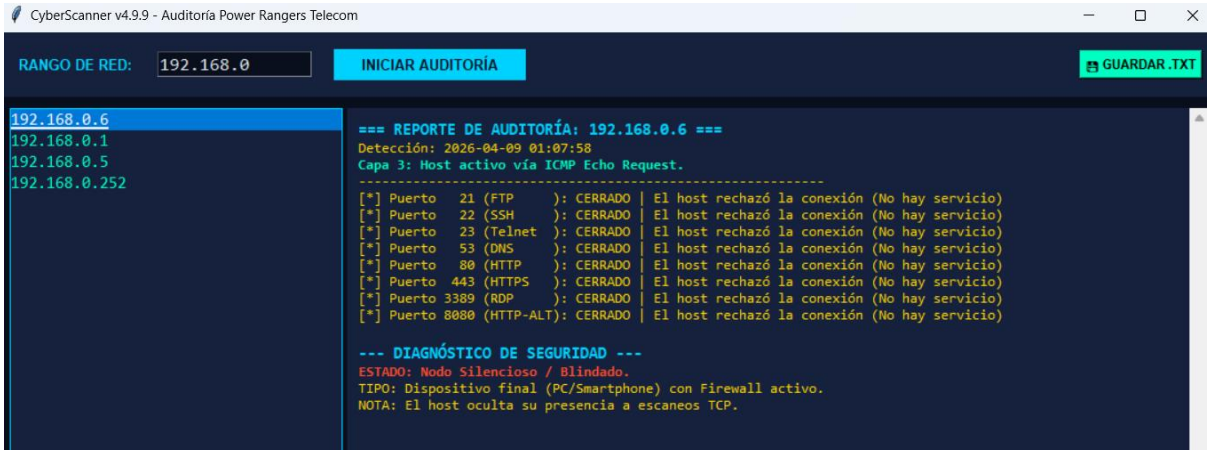
- **Inteligencia de Diagnóstico:** Se observa el sistema de clasificación automática que define al nodo 192.168.0.1 como un "Nodo Crítico" debido a la presencia de servicios de infraestructura, diferenciándolo de dispositivos finales blindados.

- **Portabilidad y Estabilidad:** Esta versión es la primera en ser totalmente compatible con la compilación a .exe en Windows. Gracias a la implementación de multiprocessing.freeze_support(), el programa puede ejecutarse de forma independiente en cualquier equipo de laboratorio sin necesidad de instalar Python.

- **Interfaz Final:** Se ha pulido la identidad visual con una paleta de colores de alto contraste que facilita la lectura de reportes técnicos extensos, cumpliendo con todos los puntos "extra" solicitados en la pauta de evaluación.

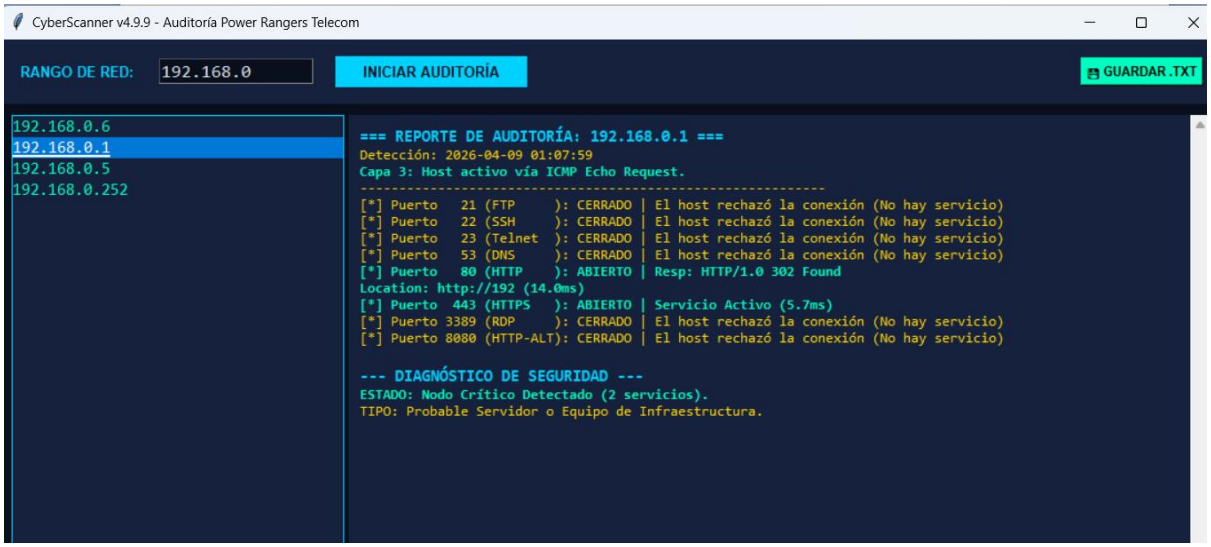
3. Evidencias de Funcionamiento

Figura 5: Escaneo de Red Local



Descripción: El escáner identifica correctamente los nodos activos en el segmento 192.168.0.0/24 e indica el diagnóstico de "Nodo Silencioso" para equipos con firewall activo.

Figura 6: Auditoría de Infraestructura (Gateway)

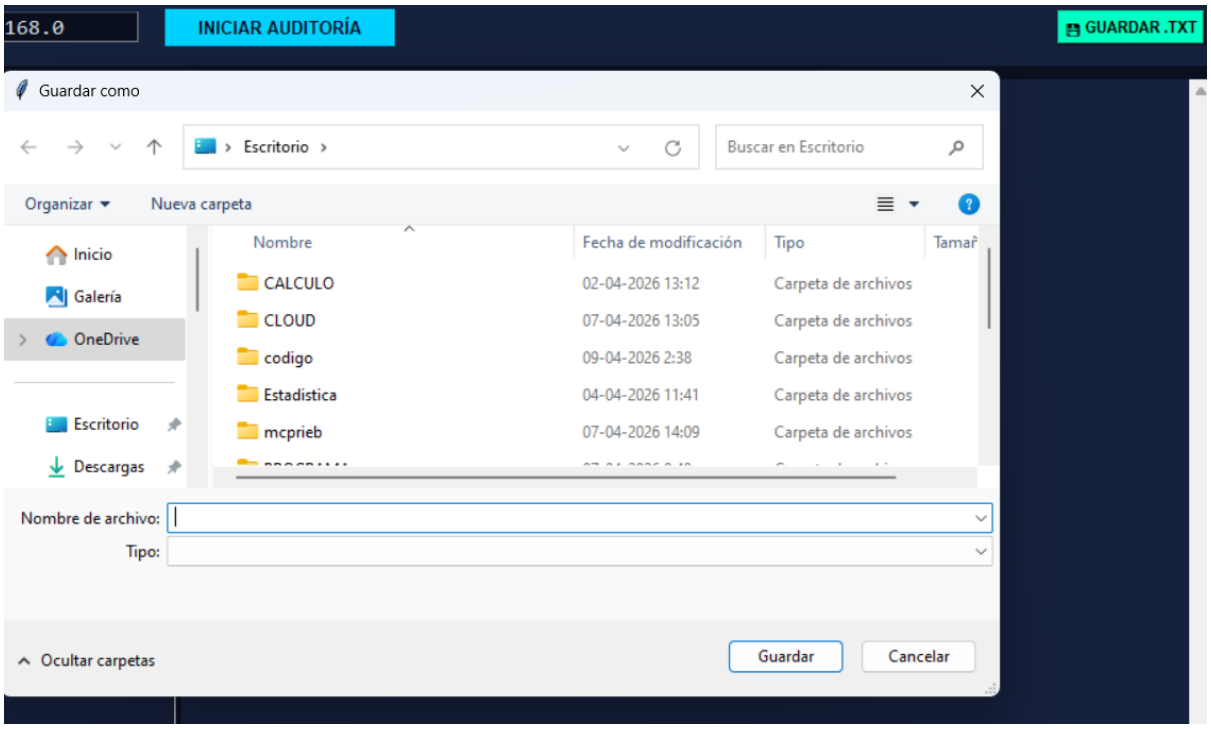


Descripción: Análisis del nodo 192.168.0.1 (Router), donde se detectan servicios HTTP y HTTPS con captura de banners.

Figura 7: Detección de Servicios Críticos

Descripción: Identificación de un puerto Telnet (23) abierto en el nodo 192.168.0.252, clasificándolo como un "Nodo Crítico".

Figura 8: Gestión de Persistencia de Datos y Exportación de Reportes



Esta captura documenta la funcionalidad de **exportación de auditoría**, uno de los pilares de la herramienta para su aplicación profesional.

- **Interacción con el Sistema Operativo:** Se observa la correcta integración de la librería filedialog de Tkinter, permitiendo al usuario seleccionar una ruta de destino local en Windows para el almacenamiento del informe.

- **Integridad del Reporte:** El archivo generado bajo el formato .txt consolida todos los hallazgos técnicos (IPs, puertos, banners y diagnósticos de seguridad).

- **Contexto Regional:** El código está configurado para incluir automáticamente una marca de tiempo con la **hora oficial de Chile (UTC-4)**, lo que garantiza la trazabilidad temporal de la auditoría frente a posibles incidentes de red.

- **Flujo de Trabajo:** Esta función cierra el ciclo de uso del software: Entrada (Rango de Red) -> Procesamiento (Escaneo Multihilo) -> Salida (Reporte Técnico Externo).

4. Gestión de Versiones (Git y Trabajo Colaborativo)

El desarrollo se gestionó mediante un repositorio de GitHub, cumpliendo con los estándares de trabajo colaborativo:

Trazabilidad: Se realizaron 22 commits documentando cada mejora y corrección.

Limpieza de Código: Uso de archivos .gitignore para excluir archivos temporales y ejecutables del repositorio.

Estructura del Repositorio:

/network_scanner: Código fuente y dependencias.

/tests: Pruebas de conectividad de socket.

README.md: Documentación técnica de acceso rápido.

Pruebas Unitarias (Carpeta /tests): En la página donde hablas de la gestión de versiones, podrías añadir un párrafo breve sobre el archivo test_basic.py que acabas de subir:

"Se incluyó una suite de pruebas unitarias básicas para validar la capacidad del sistema de abrir sockets y verificar la salida a red antes de iniciar escaneos complejos, asegurando la integridad del diagnóstico."

5. Explicación Técnica del Código (v4.9.9)

- **Importación de Librerías (Líneas 10-18):**

```
import socket, subprocess, threading, platform

import tkinter as tk

from datetime import datetime, timedelta, timezone
```

Se importan los módulos esenciales para la operación de red. socket permite la comunicación en Capa 4 (TCP), subprocess gestiona el comando Ping en Capa 3, y threading junto con multiprocessing habilitan el escaneo en paralelo.

Aplicación técnica: Sin estas librerías, el programa sería un script estático. Su inclusión permite que el software interactúe directamente con el hardware de red y el sistema operativo

- **Análisis de Capa 3: Ping Silencioso (Líneas 33-46)**

```
if sistema == "windows":

  startupinfo = subprocess.STARTUPINFO()

  startupinfo.dwFlags |= subprocess.STARTF_USESHOWWINDOW

  comando = ['ping', '-n', '1', '-w', '600', host]
```

Configura el comando ping del sistema operativo para detectar hosts activos. En Windows, usa STARTF_USESHOWWINDOW para ocultar la ventana de consola.

Aplicación técnica: Permite que el escaneo ocurra en segundo plano. Sin esto, el usuario vería cientos de ventanas negras (CMD) abriéndose y cerrándose por cada IP analizada, lo que haría la herramienta inusable.

- **Análisis de Capa 4: Sockets y Latencia (Líneas 48-61)**

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

s.settimeout(0.5)

result = s.connect_ex((host, port))}
```

Intenta una conexión TCP directa a un puerto específico. connect_ex devuelve un código de error en lugar de detener el programa si la conexión falla.

Aplicación técnica: Determina si un servicio (como Web o Base de Datos) está disponible. Al medir el tiempo entre el envío y la respuesta, el software calcula la latencia en milisegundos, indicando la calidad de la conexión.

- **Banner Grabbing e Identificación (Líneas 63-71)**

s.send(b'HEAD / HTTP/1.0\r\n\r\n')

banner = s.recv(512).decode().strip()[:40]

Si el puerto está abierto, el código envía una cabecera genérica para forzar al servidor a responder con su nombre o versión.

Aplicación técnica: Convierte un simple escáner en una herramienta de reconocimiento. Permite saber, por ejemplo, si el puerto 80 es un servidor Apache, Nginx o un dispositivo embebido, facilitando el inventario de activos.

- **Diagnóstico de Seguridad: Firewall/Filtrado (Líneas 73-82)**

elif result in [11, 35, 110, 10060]: # Códigos de Timeout

return "ALERTA", "FILTRADO | [!] Paquete descartado por Firewall (DROP)"

Clasifica los errores de conexión. Si el error es un *Timeout* (código 10060 en Windows), se asume que el host ignoró el paquete.

Aplicación técnica: Logra diferenciar un puerto "Cerrado" (el host rechaza la conexión) de uno "Filtrado" (el host o un Firewall intermedio descartan el paquete). Esto es vital para evaluar la seguridad perimetral de la red.

- **Control de Concurrencia: El Semáforo (Líneas 95-97)**

self.semaphore = threading.Semaphore(40)

Aplicación técnica: Evita el colapso de la CPU y que el sistema operativo identifique el escaneo como un ataque de denegación de servicio (DoS). Permite un equilibrio entre velocidad extrema y estabilidad del PC

- **Motor de Escaneo Paralelo: Hilo Maestro (Líneas 128-142)**

for i in range(1, 255):

t = threading.Thread(target=self.worker_limitado, args=(ip,), daemon=True)

t.start()

Crea un hilo independiente para cada una de las 254 direcciones IP del segmento /24.

Aplicación técnica: Permite que la interfaz gráfica (GUI) no se "congele" mientras se realiza el escaneo. El usuario puede seguir interactuando con el programa mientras los resultados aparecen en tiempo real.

- **Lógica de Auditoría y Diagnóstico (Líneas 148-180)**

```
if encontrados > 0:
    reporte.append(("TIPO: Probable Servidor o Equipo de Infraestructura.\n", "INFO"))
else:
    reporte.append(("TIPO: Dispositivo final con Firewall activo.\n", "INFO"))
```

Aplicación técnica: Transforma datos brutos en información de inteligencia. El software no solo entrega números, sino que ofrece una conclusión técnica sobre si el equipo es un servidor crítico o un terminal de usuario protegido.

- **Persistencia y Seguridad de Datos (Líneas 178-200**)

```
with self.db_lock:
    self.host_database[ip] = reporte
```

Uso de un Lock (bloqueo) para escribir en la base de datos interna y la función de exportación a .txt.

Aplicación técnica: Garantiza la integridad de los datos al evitar que dos hilos escriban al mismo tiempo (condición de carrera). La función de guardado consolida todos los hallazgos en un documento formal de auditoría.

6. Conclusión

El proyecto CyberScanner v4.9.9 cumple con los requisitos de funcionalidad, calidad de código y documentación exigidos. La implementación de extras como la interfaz gráfica y el diagnóstico de firewall posicionan esta herramienta como una solución robusta para auditorías básicas de red.